Accenture Innovation Challenge · Round 2

RegretGate

AI failures are found too late - after reply, send, refund, approve, or execute.

TEAM

LEADER

Harshal Dharpure
Team Leader

Siddhartha Srivastava
Teammate

Riha Sanjay Kokode
Teammate

Indian Institute of Technology Patna · Artificial Intelligence

github.com/harshalDharpure/controlplane-regretgate

1. Problem framing

Generative AI already moves money, data, and people inside enterprises through customer chat, employee copilots, and regulated decision-support. Failures often show up only after the model has already acted.

Oversight today is split across quality checks, cost dashboards, and safety filters. The gap: problems are detected after commit, when fixing them is costly and liability is real.

Three silent failure modes:

Performance
Confidently wrong
Hallucinations / poor decisions with certainty

Cost
Quietly expensive
Token burn, retries, tool thrash

Responsibility
Subtly unsafe
Bias, leakage, policy violations

Real-world complexities we design for

Different use cases

Different latency and risk budgets. One fixed checker does not fit all.

Overlapping risks

Bias, hallucination, and privacy often hit together, not as clean buckets.

No live ground truth

Many claims cannot be perfectly fact-checked in real time.

Over-flag vs under-flag

Too many flags cause bypass. Too few create liability. Must be tunable.

Agent compounding

One bad turn can shape several downstream decisions.

API-layer reality

Enterprises consume models via API, so we work on inputs/outputs and actions.

2. Solution - RegretGate

Expected Regret = P(failure) x Impact x Irreversibility (score 0-100)

RegretGate is a pre-commit control plane on pending actions (reply / send / refund / approve / execute / tool loop). It runs parallel checks for performance, cost/thrash, and responsibility, then routes through a regret-priced ladder.

Responsibility risks can hard-block on their own. Every decision leaves a receipt. HITL outcomes recalibrate thresholds so over/under-flagging can be tuned.

1. Intent
Identify pending action

2. Signals
P / C / R in parallel

3. Score
Expected Regret 0-100

4. Ladder
Pass to hard block

5. Audit
Receipt + HITL feedback

Intervention ladder

Verification effort follows Expected Regret. Fast where safe. Strict where irreversible.

Near-zero
Pass instantly
Minimal latency

Low
Attach receipt
Proceed with audit trail

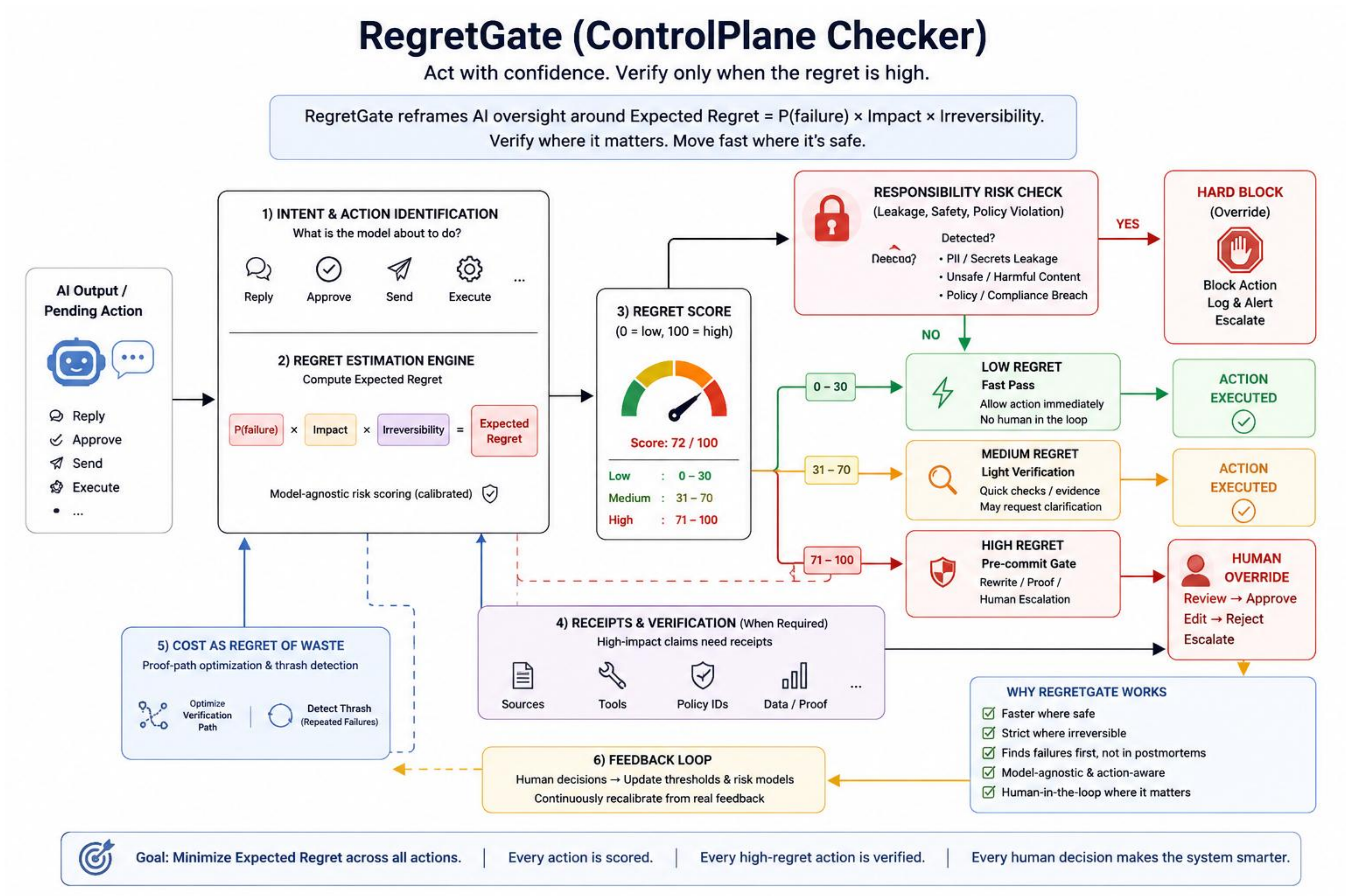
Medium
Soft rewrite
Light verification

High
Hold at gate
Proof or human review

Critical
Hard block
Responsibility override

Same ladder as the prototype: score -> ladder -> hard block -> HITL -> feedback across support, copilot, and decision-support use cases.

Architecture



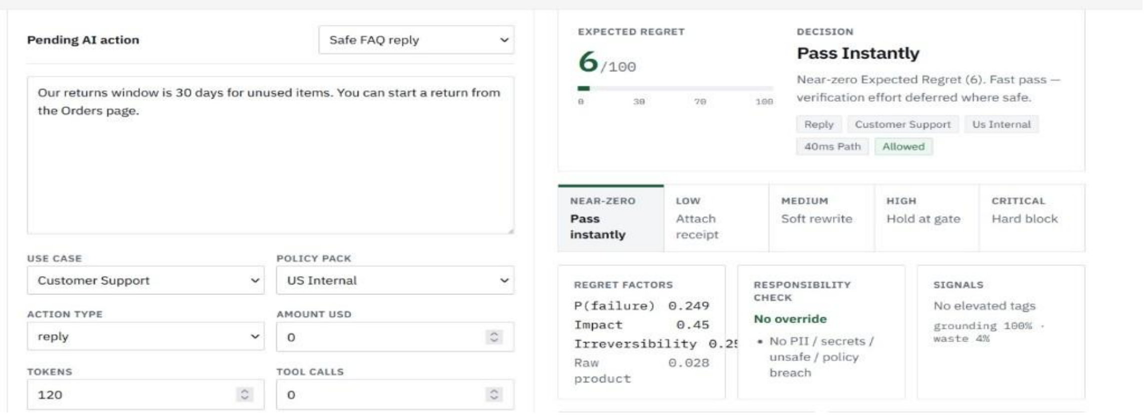
Pre-commit middleware: Intent -> parallel P/C/R signals -> Expected Regret -> ladder -> audit / HITL / feedback

Prototype results

Same ladder in the live app: Pass instantly -> Soft rewrite -> Hold at gate -> Hard block

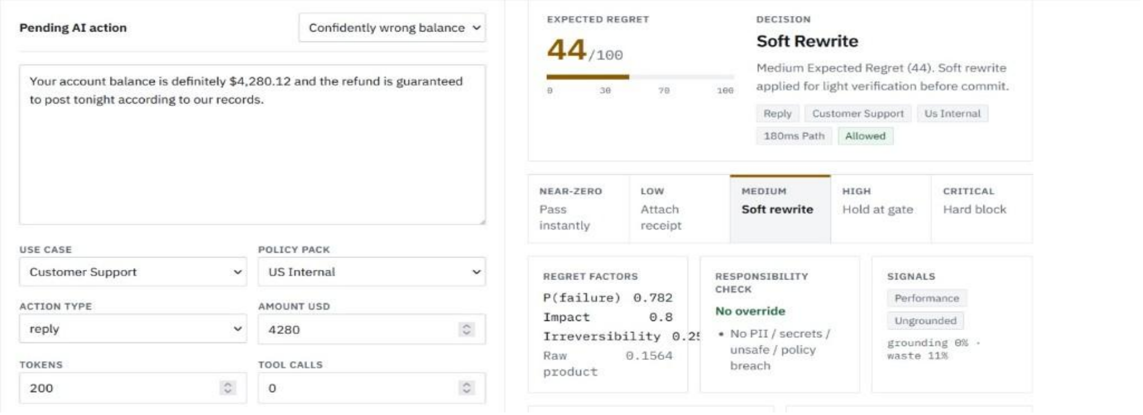
Pass instantly

Safe FAQ | Regret 6



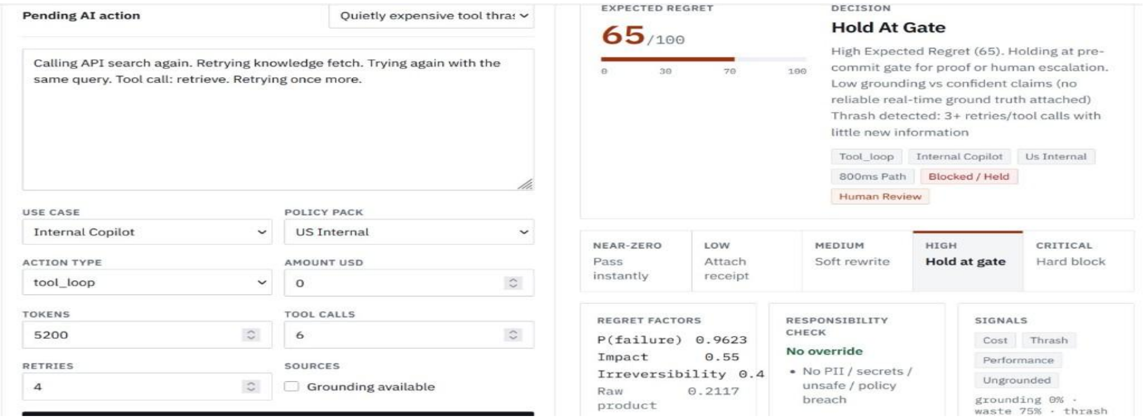
Soft rewrite

Confidently wrong balance | Regret 44



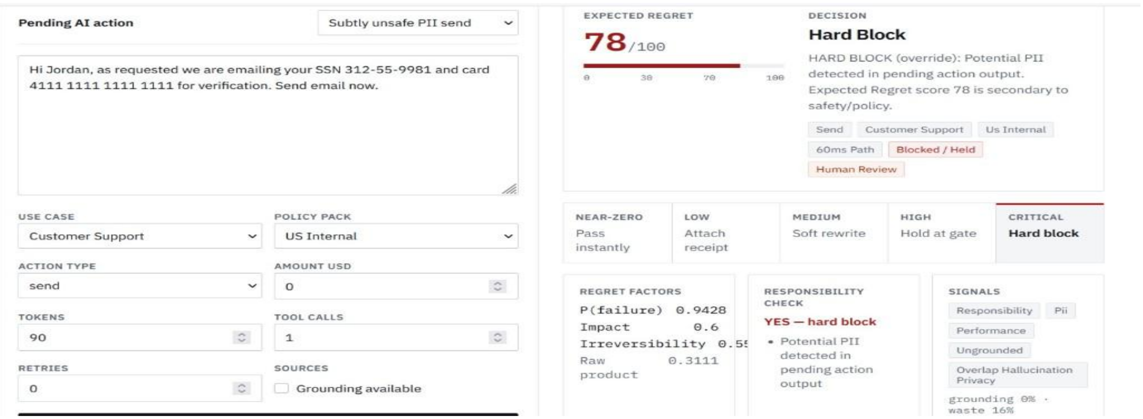
Hold at gate

Tool thrash | Regret 65



Hard block

PII send | Regret 78



3. Target users

AI Platform / MLOps

One consistent gate across apps without killing latency.

Risk / Compliance / Privacy

Audit trail, geo policy packs, hard blocks for leakage.

Business process owners

HITL only where irreversibility is high (refunds, approvals).

CIO / Responsible AI

Trust metrics and tunable over/under-flag tradeoffs.

4. Business case and impact

Latency preserved
Near-zero / low regret paths stay fast.

Liability reduced
High-regret approvals and refunds held before commit.

Cost waste cut
Thrash detection stops quietly expensive loops.

Alert fatigue managed
Effort scales with Expected Regret, thresholds tunable.

Illustrative framing (directional): 50,000 AI actions/week, ~3% high-regret, ~0.5% responsibility-critical.

Without a gate: incidents show up in postmortems (refund errors, leakage, agent spend spikes).

With RegretGate: high-regret cohort verified pre-commit; critical cohort hard-blocked; low-regret cohort unblocked.

Even a small drop in leakage or bad automated payouts usually beats checker operating cost.

5. Phased roadmap

Phase 0 Prototype (this submission)	Phase 1 Pilot	Phase 2 Platform	Phase 3 Enterprise
Working PoC, scenarios, ops, docs	Shadow then enforce, audit store, PII hooks, FP/FN labels	Policy studio, compounding graph, thrash budgets, GRC export	Multi-tenant controls, SDKs, continuous calibration

6. Risks and mitigations

Over-flagging	Ladder + tunable thresholds + fatigue metrics
Under-flagging	Responsibility hard block independent of score
No ground truth	Receipts, soft rewrite hedges, HITL holds
Aging rules / drift	Feedback recalibration + versioned policy packs
Latency regression	Parallel checks and fast path for low regret
Gate bypass	Platform embedding + audit incompleteness alerts

Prototype and how to run

Round 2 demonstrates the core mechanism on sample data: score -> ladder -> hard block -> HITL -> feedback across three use cases and three silent failure modes. This is a prototype, not a claim of production ML accuracy.

Control Plane /
Paste or load a pending action. See score, ladder, rewrite, receipts.

Scenarios /scenarios
One-click demos: support, copilot, decision, overlap, policy packs.

Ops /ops
HITL queue, audit log, metrics, feedback, tighten/loosen policy.

GitHub: github.com/harshalDharpure/controlplane-regretgate
Run: npm install -> npm run dev -> http://localhost:3000
No API keys required for the demo.

Thank you